

2025/8/28

# 数据奥德赛：

## 基于 Python 的短视频平台数据特征分析和 多种模型预测

### 摘要

本文聚焦快手短视频平台数据展开研究。基于 2022 年 4 - 5 月视频、用户等数据，先剖析数据类型与结构，探究视频数据特征，含类型占比、时长分布等，构建打分模型分级视频质量，分析上传类型影响；多角度解析用户行为，包括用户活跃度、身份社交关联等角度；借 SkLearn 搭建 KNN、K - mean 及混合机器学习推荐模型，经流程实现视频推荐；构建混合预测模型，含 SARIMA、随机森林，完成视频每小时点赞量 48 小时预测。为平台数据研究、运营及模型应用提供实践参考，也指出数据量等可优化方向。

秦子宸

2025/8/27

zichenq2@uci.edu

# 目录

|                                               |            |
|-----------------------------------------------|------------|
| 一、数据类型和数据基本结构 .....                           | 1          |
| 二、视频数据基本特征与视频质量模型 .....                       | 2          |
| (一) 统计分析结果 .....                              | 2          |
| 1. 不同类型视频的数量和占比 .....                         | 2          |
| 2. 每个类别的视频数量 (top5) 和上传时间的分布 .....            | 3          |
| 3. 视频时长的分布情况 .....                            | 4          |
| 4. 常见分辨率的比例 .....                             | 5          |
| 5. 背景音乐 ID 的使用频率和背景音乐类型的分布 .....              | 6          |
| (二) 建模分析结果 .....                              | 6          |
| 1. 打分体系建模效果展示 .....                           | 7          |
| 2. 不同上传类型对用户体验的影响 .....                       | 7          |
| (三) 讨论与总结 .....                               | 错误! 未定义书签。 |
| 三、多角度分析用户行为特征 .....                           | 9          |
| (一) 包含角度 .....                                | 9          |
| (二) 图形具象化结果和报告展示 .....                        | 9          |
| 1. 用户活跃度分析。 .....                             | 9          |
| 2. 用户身份与社交特征的关系 .....                         | 9          |
| 3. 研究注册时长对用户行为特征的影响 .....                     | 11         |
| 4. 用户社交行为的驱动因素分析 .....                        | 13         |
| 四、基于 SkLearn 的短视频平台机器学习推荐模型 .....             | 15         |
| (一) 使用模型 .....                                | 15         |
| (二) 模型搭建 .....                                | 16         |
| 1. K-mean 模型 .....                            | 16         |
| 2. KNN 模型 .....                               | 18         |
| 3. KNN 和 K-mean 结合使用的预测模型 .....               | 21         |
| (三) 结论 .....                                  | 23         |
| (四) 讨论与总结 .....                               | 23         |
| 五、视频每小时点赞量混合预测模型 .....                        | 24         |
| (一) 模型核心目标 .....                              | 24         |
| (二) 使用模型和模型介绍 .....                           | 24         |
| (三) 模型构建主要流程 .....                            | 25         |
| 步骤 1: 数据获取与分割 (get_data 函数) .....             | 25         |
| 步骤 2: 混合模型训练与预测 (train_hybrid_model 函数) ..... | 26         |
| 步骤 3: 结果可视化与输出 (plot_results 函数 + 主程序) .....  | 28         |
| (四) 讨论与总结 .....                               | 29         |
| 附录 .....                                      | 30         |

## 一、数据类型和数据基本结构

本数据集主要来自于“快手短视频”2022 年 4 月~5 月两个月期间的视频数据、用户数据、平台日志等，包含日志（“log\_standard”）、用户数据（“user\_features\_basic”）、视频信息（“video\_featrue\_basic\_pure”）和视频统计信息（“video\_featrue\_statistic”）这四个数据表。

在本次研究中，主要使用了视频基础特征数据集（“video\_feature\_basic\_pure”）和日志（“log\_standard”），数维度主要包含视频 id、作者 id、视频类型、上传方式、上传时间、视频持续时间、音乐 id、分辨率、标签等，数据量超过 7000。此外还使用了视频统计数据集（“video\_featrue\_statistic”），主要是视频完播率、点赞数等。

数据预处理主要包括去除空值（NULL），以及修改数据类型等。其中，视频类型为广告的数据在 music\_id 和 muic\_type 栏皆为空值，可能是因为没有统计或者视频类型为照片造成的，所以在预处理中全部改为了 0，方便后续进行研究和计算。

## 二、视频数据基本特征与视频质量模型

本次研究的统计部分主要使用 MySQL 的查询，以及 Matplotlib 库。

在统计视频时长的时候，考虑到不同时长区间里，人们对时长差的敏感度不一样（比如短视频中 30 秒与 40 秒的差距，比长视频里 30 分钟与 35 分钟的差距更突出），所以对比时最好按视频长短分段进行，这样会更合理。所以这里首先定义了时间区间（0~600s 是每 30s 一个区间，600s~3600s 是每 5 分钟一个区间），再进行视频时长区间分布统计，从而更好的展现分布效果。

在统计视频分辨率时，由于部分视频是竖版的，存在分辨率完全一样，但是长和宽对调的情况，盲目进行统计会多输出另外一中分辨率。这里采用的方式是按大小将宽和高进行重新排序，保证较长的边在前，短边在后，从而确保相同像素的分辨率得到相同标识。

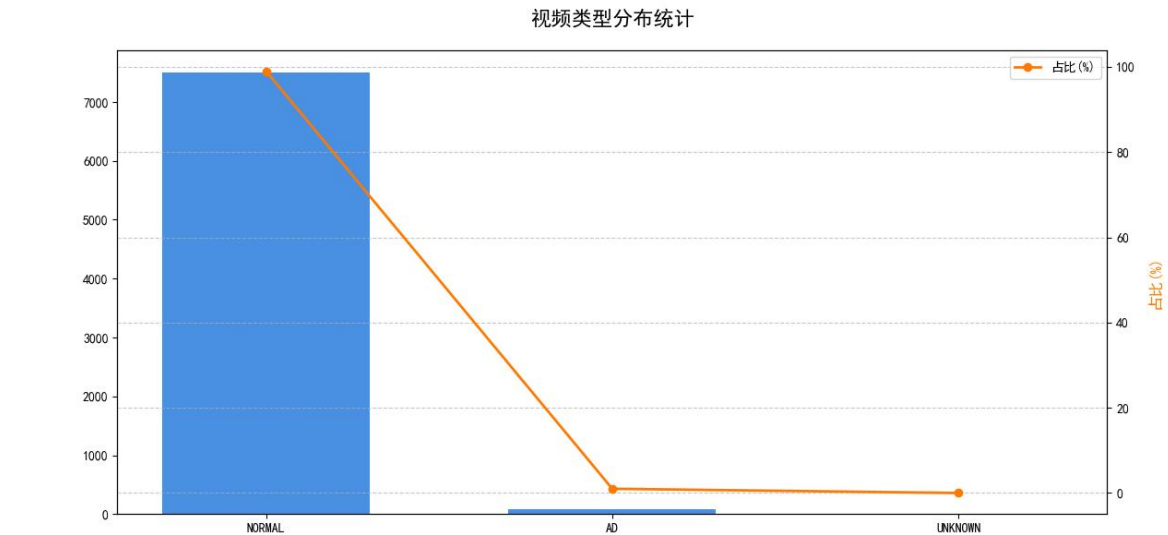
针对视频质量等级，主要建了一个打分模型，根据不同的数据维度对该视频的整体质量进行评估。在研究中，关联查询了视频的分辨率、点赞数、完播量、评论数、已播率等数据，并对每种数据维度构建了打分标准，保证每个区间的增长幅度相同，比如点赞数从 0~100 得 1 分，100~300 得 2 分，300~600 得 3 分，600~1200 得 4 分（每个区间大小的增长幅度为上个区间上限的 100%）。随后通过 `pd.cut` 的方法，将数据进行分段，并且将每段种的数据最后一列（`score` 列）加上对应的分数。再算完所有分数后，按照一定的等级，将不同的视频质量进行分级。质量最高的视频为 1 级视频，质量最低的视频为 4 级视频。每个等级的分数区间是 0~7 分，7~14 分，14~21 分，21 分以上。再计算完后，再通过查询原始数据，将打分后的数据和原始数据利用 `pd.merge` 的方法整合到一起，从而分析不同上传途径的视频质量。

### （一）统计分析结果

#### 1. 不同类型视频的数量和占比

总共有以下 3 种，数量和占比如下：

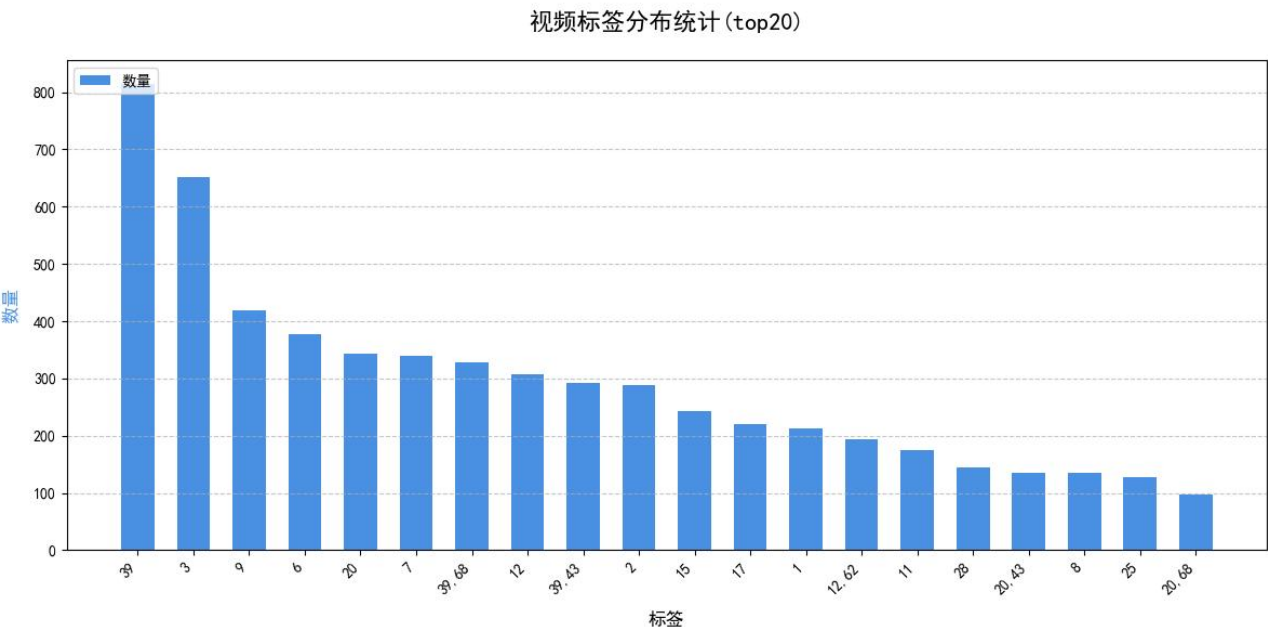
|   | video_type | count | percent |
|---|------------|-------|---------|
| 0 | NORMAL     | 7506  | 98.98   |
| 1 | AD         | 76    | 1.00    |
| 2 | UNKNOWN    | 1     | 0.01    |



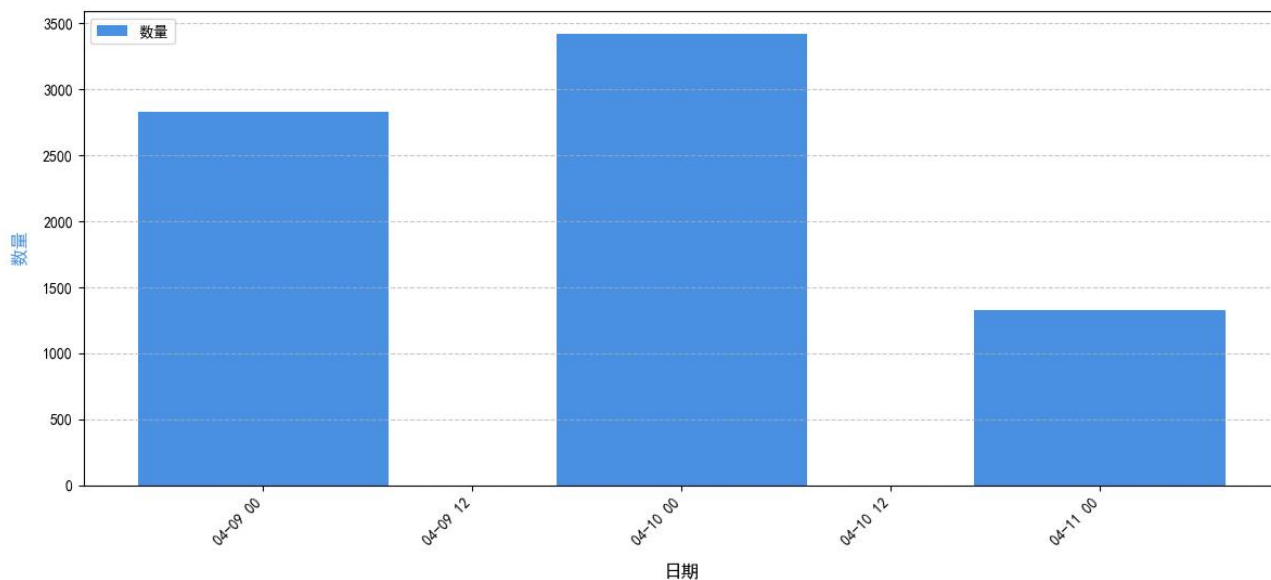
根据数据可知，在本数据集中，绝大部分（98.98%）都是常规视频，只有少数的广告和一个未知视频。未知视频可能是因为统计失误或者上传的网络问题，在误差范围内（小于等于 0.05%），所以可以忽视。

2. 每个类别的视频数量（top5）和上传时间的分布

|      | tag | count |
|------|-----|-------|
| 0    | 39  | 815   |
| 6    | 3   | 651   |
| 4    | 9   | 418   |
| 136  |     | 378   |
| 2720 |     | 344   |
| 3    | 7   | 340   |

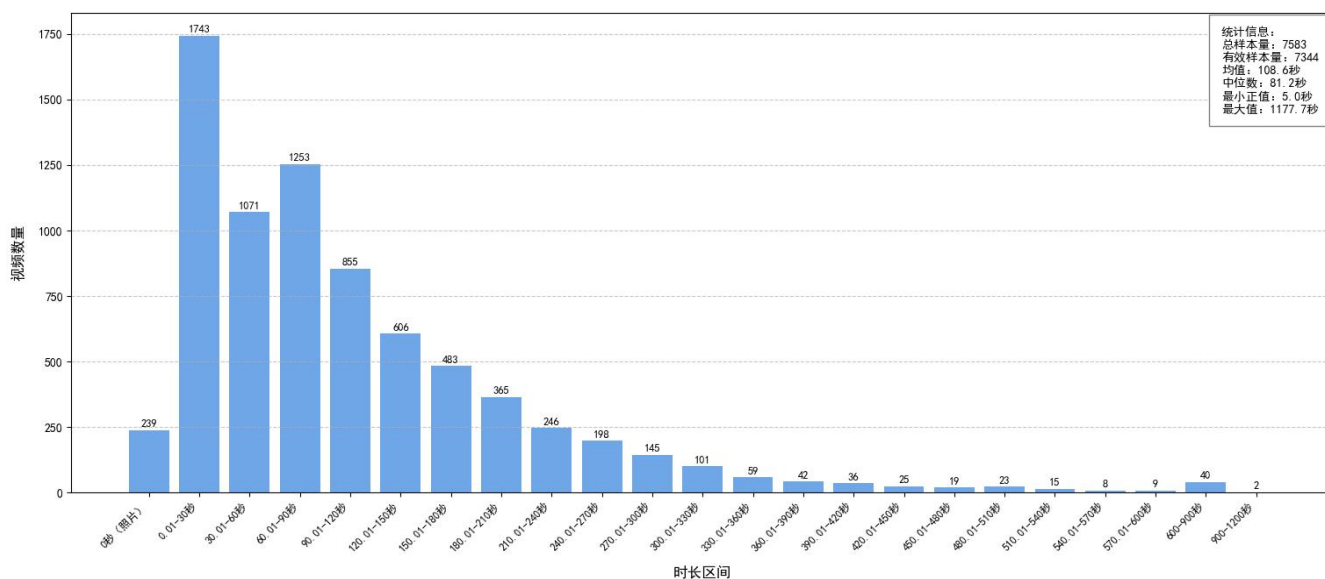


视频发布时间分布统计



### 3. 视频时长的分布情况

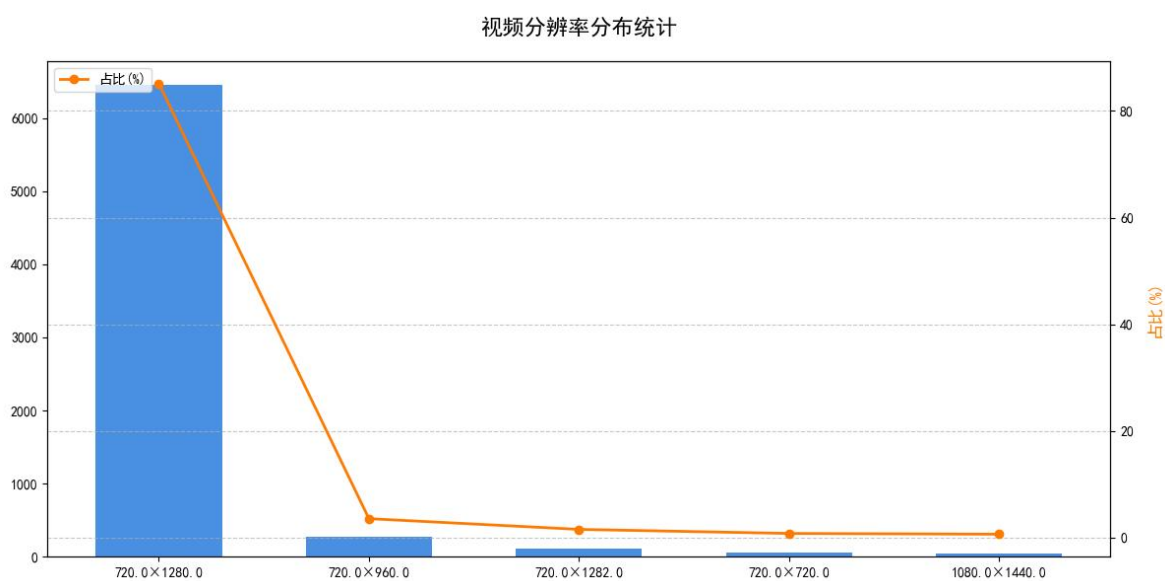
视频时长区间分布



由图可见，上传的视频中最多的短视频（0~30 秒），其次是 1 分钟左右的中长度视频。该平台大部分视频都是短视频，越长的视频越少。

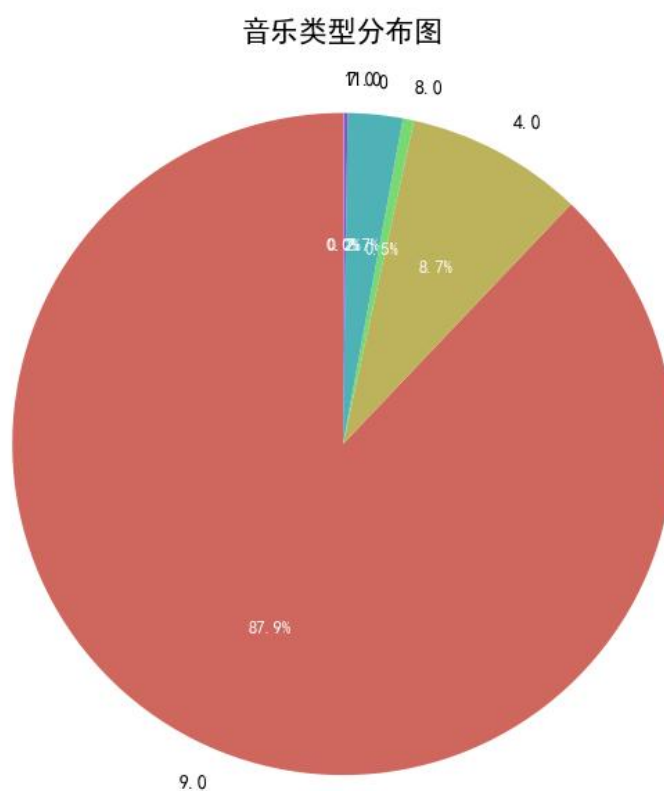
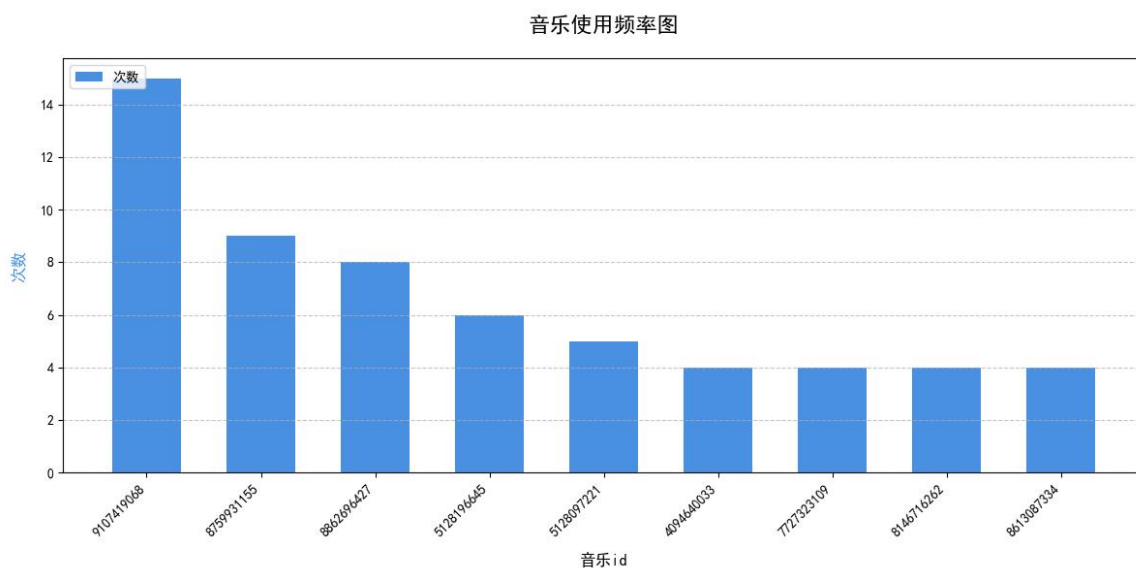
## 4. 常见分辨率的比例

|   | normalized_resolution | count | 占比(%) |
|---|-----------------------|-------|-------|
| 0 | 720.0×1280.0          | 6457  | 85.15 |
| 1 | 720.0×960.0           | 269   | 3.55  |
| 2 | 720.0×1282.0          | 117   | 1.54  |
| 3 | 720.0×720.0           | 59    | 0.78  |
| 4 | 1080.0×1440.0         | 48    | 0.63  |



本数据集绝大部分视频的分辨率都是 720\*1280，而有少部分的分辨率在 720\*960 以下（不足 10%），说明绝大部分的上传都是长方形高画质的视频。

## 5. 背景音乐 ID 的使用频率和背景音乐类型的分布



使用最多的音乐类型是 9 类，其次是 4 类。使用的最多的音乐 id 是 9104719068，断档领先于其他的类型。

## (二) 建模分析结果

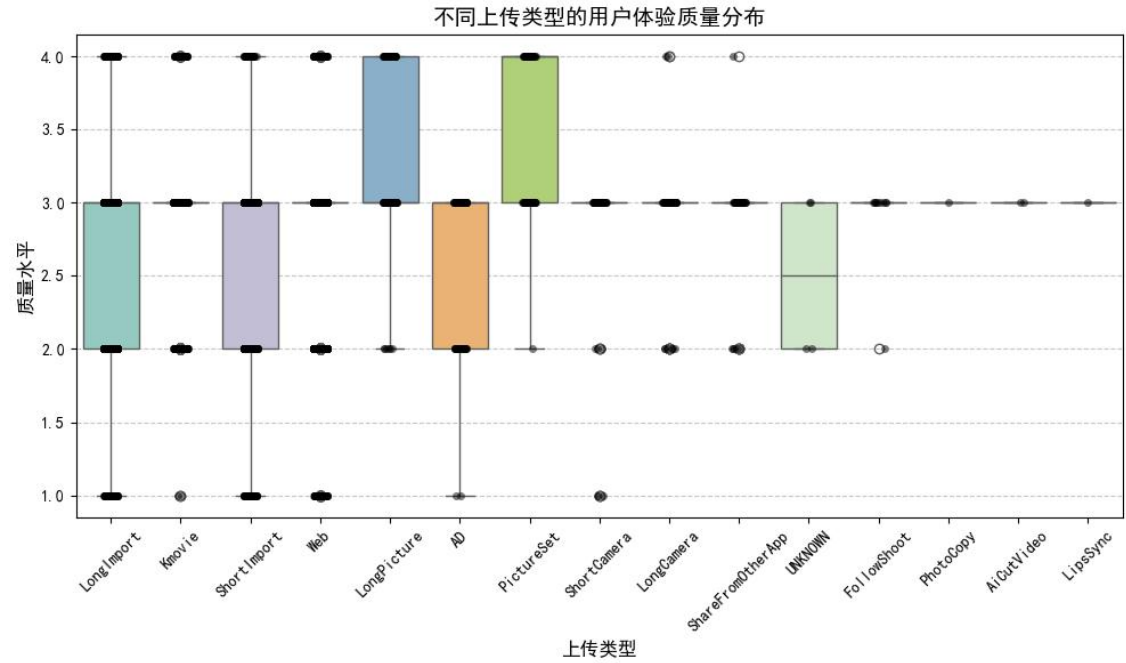
## 1. 打分体系建模效果展示

最右侧是视频等级，质量最高的视频为 1 级视频，质量最低的视频为 4 级视频。每个等级的分数区间是 0~7 分，7~14 分，14~21 分，21 分以上。

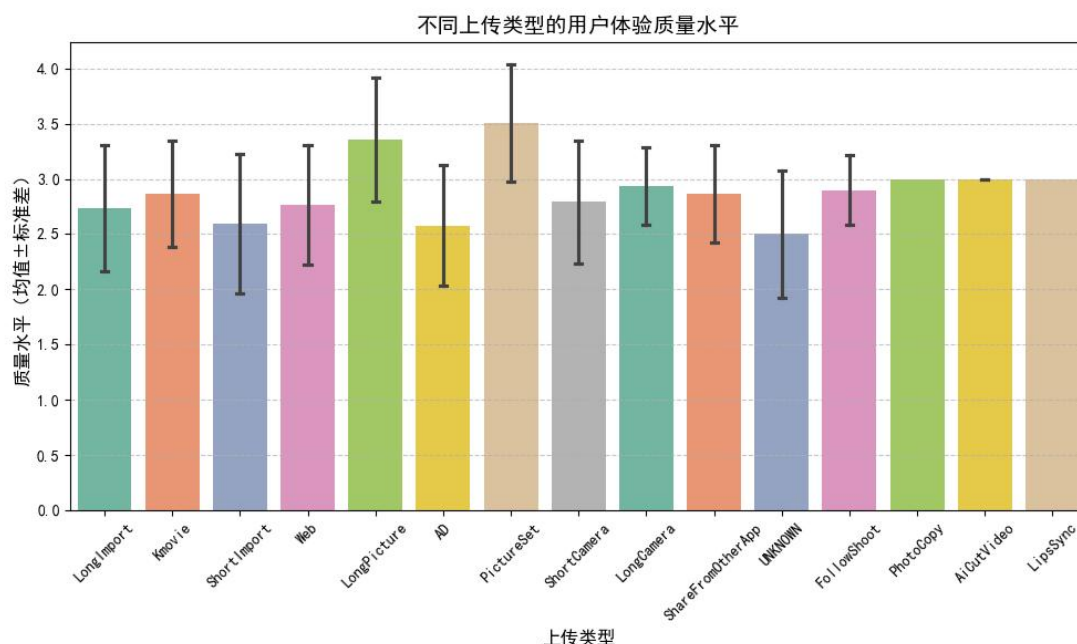
| video_id | video_duration | like    | complete_play | share        | commnet_user_number | play_progress | resolution | quality_level |   |
|----------|----------------|---------|---------------|--------------|---------------------|---------------|------------|---------------|---|
| 0        | 0              | 874.33  | 6.470588      | 195.254902   | 0.058824            | 0.196078      | 0.117407   | 1280.0x720.0  | 3 |
| 1        | 1              | 2180.66 | 38.161765     | 130.507353   | 1.698529            | 2.345588      | 0.064726   | 1280.0x720.0  | 3 |
| 2        | 2              | 92.33   | 6.626374      | 80.208791    | 0.197802            | 1.208791      | 0.238342   | 1280.0x720.0  | 3 |
| 3        | 3              | 164.33  | 5.402778      | 115.930556   | 0.013889            | 0.041667      | 0.217354   | 1280.0x720.0  | 3 |
| 4        | 4              | 387.66  | 457.235955    | 1329.078652  | 24.865169           | 38.202247     | 0.308945   | 1280.0x720.0  | 2 |
| 5        | 5              | 3580.16 | 137.287356    | 496.954023   | 0.965517            | 3.965517      | 0.100842   | 720.0x960.0   | 3 |
| 6        | 6              | 1145.00 | 93.811475     | 678.368852   | 0.426230            | 2.090164      | 0.169716   | 720.0x960.0   | 3 |
| 7        | 7              | 0.00    | 13.643564     | 5.693069     | 0.504950            | 0.237624      | 0.031681   | 1440.0x720.0  | 4 |
| 8        | 8              | 1503.66 | 1711.177778   | 3406.238889  | 17.933333           | 53.577778     | 0.128284   | 1280.0x720.0  | 2 |
| 9        | 9              | 619.33  | 18.303030     | 192.659091   | 2.045455            | 0.439394      | 0.107336   | 1280.0x720.0  | 3 |
| 10       | 10             | 608.16  | 14.127660     | 73.957447    | 0.063830            | 0.425532      | 0.115645   | 1280.0x720.0  | 3 |
| 11       | 11             | 1374.80 | 57.253333     | 1669.473333  | 0.360000            | 0.433333      | 0.169997   | 1280.0x720.0  | 3 |
| 12       | 12             | 202.00  | 14.910256     | 166.102564   | 0.076923            | 0.282051      | 0.152695   | 1280.0x720.0  | 3 |
| 13       | 13             | 1748.39 | 2.485437      | 6.582524     | 0.203883            | 0.067961      | 0.026949   | 1280.0x720.0  | 3 |
| 14       | 14             | 253.66  | 8.516393      | 60.737705    | 0.073770            | 0.385246      | 0.085634   | 1280.0x720.0  | 3 |
| 15       | 15             | 194.64  | 26.303030     | 217.848485   | 1.689394            | 3.901515      | 0.159418   | 1280.0x720.0  | 3 |
| 16       | 16             | 1334.80 | 41.500000     | 973.009091   | 0.363636            | 1.336364      | 0.164211   | 1280.0x720.0  | 3 |
| 17       | 17             | 337.00  | 326.427778    | 197.600000   | 2.938889            | 7.288889      | 0.135566   | 1280.0x720.0  | 3 |
| 18       | 18             | 1276.07 | 1.343284      | 85.716418    | 0.014925            | 0.059701      | 0.141039   | 1280.0x720.0  | 3 |
| 19       | 19             | 183.66  | 432.765363    | 10489.284916 | 1.217877            | 32.491620     | 0.355721   | 1280.0x720.0  | 2 |

## 2. 不同上传类型对用户体验的影响

不同上传方式的视频质量箱线图如下（越低的箱视频质量越高）：



本图可见，通过 LongImport、ShortImport 上传的视频水平整体比较高，但是上下限差距很大。而通过 LongPicture 和 PictureSet 上传的视频质量都不高，评分都比较低，绝大部分都在 3~4 级之间。上传方式中，通过 PhotoCopy、AiCutVideo、LipsSync 上传的视频质量最差，最高等级为 3 级，也只有 3 级。



本图是根据不同种类上传方式的视频等级均值绘制而成，包括标准差。从本土可见广告（AD）的视频质量最高，而 LongPicture 和 PictureSet 最低，且浮动很大（标准差大）。而通过 LongCamera 和 FollowShot 上传的视频整体不算高，但是整体质量最均很，浮动最小（标准差小）。

### （三）讨论与总结

本研究主要包含多个数据特征分析以及一个打分体系模型，针对来自短视频平台短期内的数据进行了初步的研究，分析该平台视频的基本数据特征，从而能够加强读者对于本数据和该平台视频的主要认知，为以后的数据研究做好了铺垫。

本研究的主要优点在于主要通过统计的方式进行研究，并且包含大量可视化图像，让研究结果清晰易读。此外，模型的打分区间和视频时间区间都采用分段的方式，能让不同时长维度下的打分标准与用户实际感知更贴合，避免因跨区间对比导致的认知偏差，符合人们对于比例对比和数值对比之间的认知差异，让结构更加合理。

本研究依旧可以改善的地方在于数据量相对较少，所以有些数据不够显著，甚至存在误差较大的情况（比如本数据集种所有的视频都是在于不可见状态，无法进行分析）。在以后的研究中，首先应该增加数据量，并且可以改善分析方法，将本数据集中更多的数据特点和统计量进行展示。

## 三、多角度分析用户行为特征

### （一）包含角度

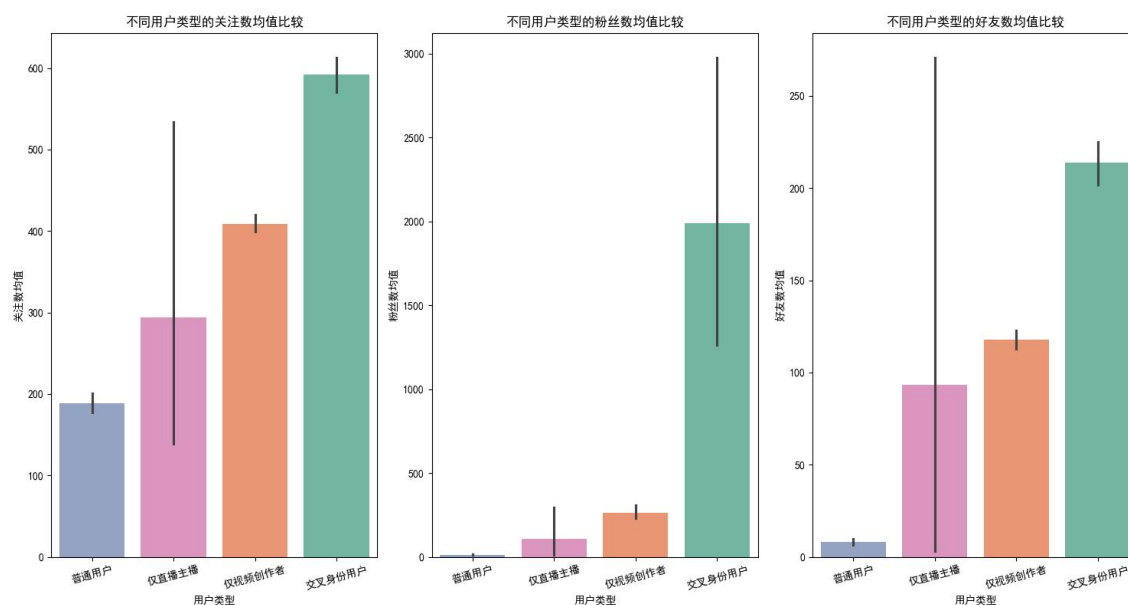
- 用户活跃度特征分析
- 用户身份与社交特征的关联
- 研究注册时长对用户行为特征的影响
- 用户社交行为的驱动因素分析

### （二）图形具象化结果和报告展示

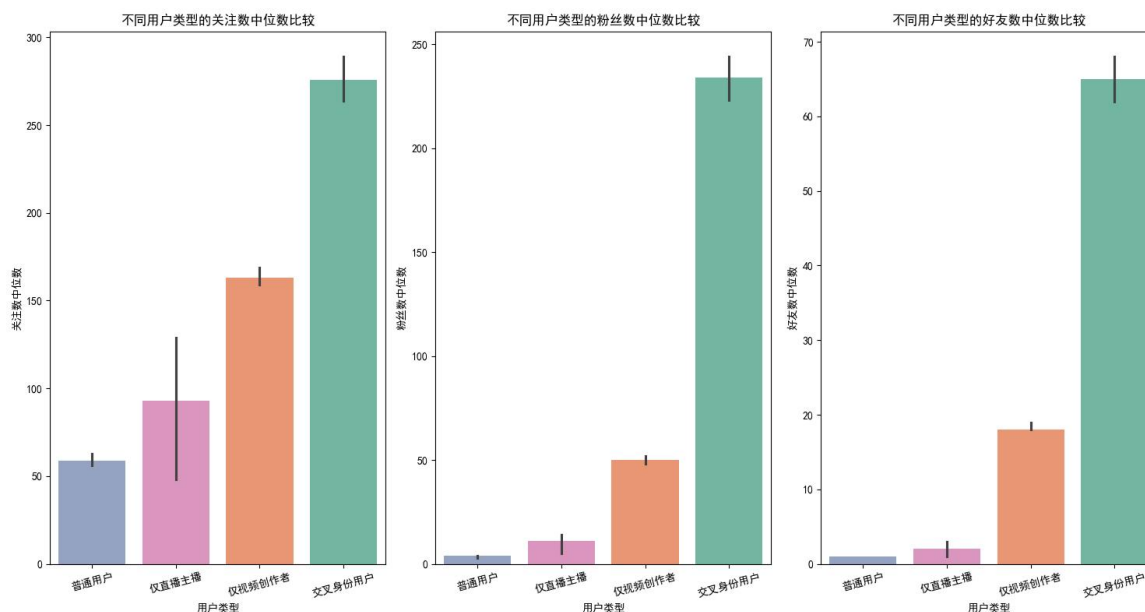
#### 1. 用户活跃度分析。

本部分由于数据量不够，展现用户全是低活跃度情况，没法进行分析，所以暂且跳过。

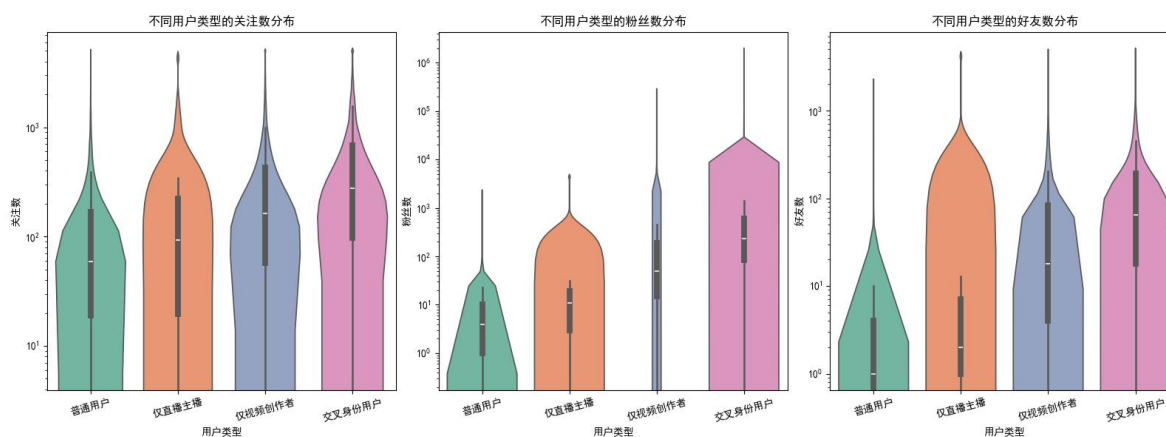
#### 2. 用户身份与社交特征的关系



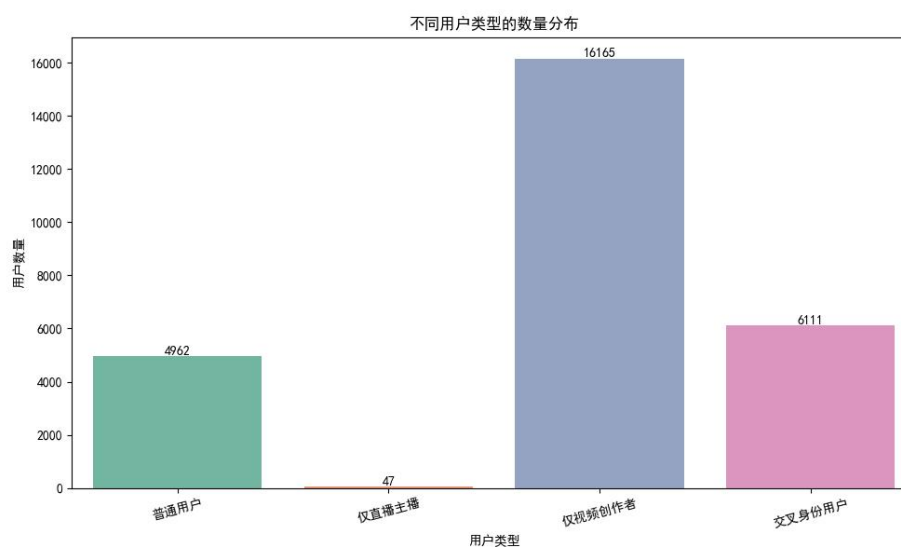
这三张图分别呈现不同用户类型（普通用户、仅直播主播、仅视频创作者、交叉身份用户）的关注数、粉丝数、好友数均值对比。关注数上，交叉身份用户均值最高，普通用户最低；粉丝数中，交叉身份用户均值远超其他，普通用户近乎为 0；好友数里，交叉身份用户均值居首，普通用户依旧最低。误差线显示数据离散情况，反映出不同身份用户社交数据差异。



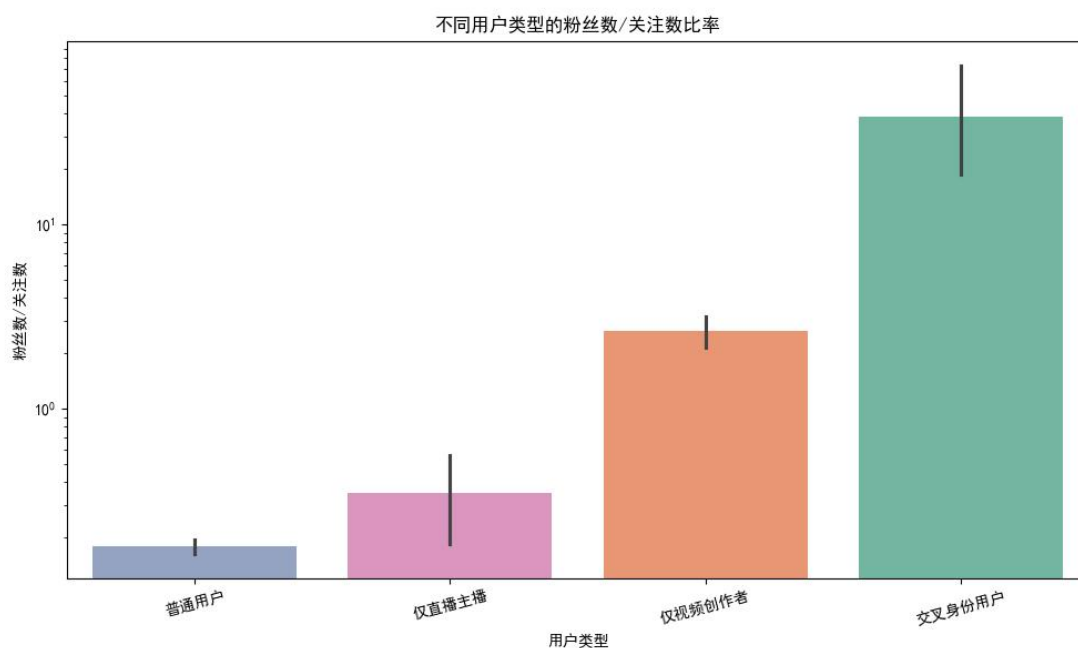
交叉身份用户在三项数据中位数上均大幅领先，仅直播主播、仅视频创作者居中，普通用户数值最低，体现不同身份用户社交数据差异。



交叉身份用户的三项数据在分布上集中于更高区间，普通用户多集中在低数值段，仅直播 / 视频创作者则介于两者之间，核密度曲线的峰值位置和跨度清晰体现了这种分层差异。

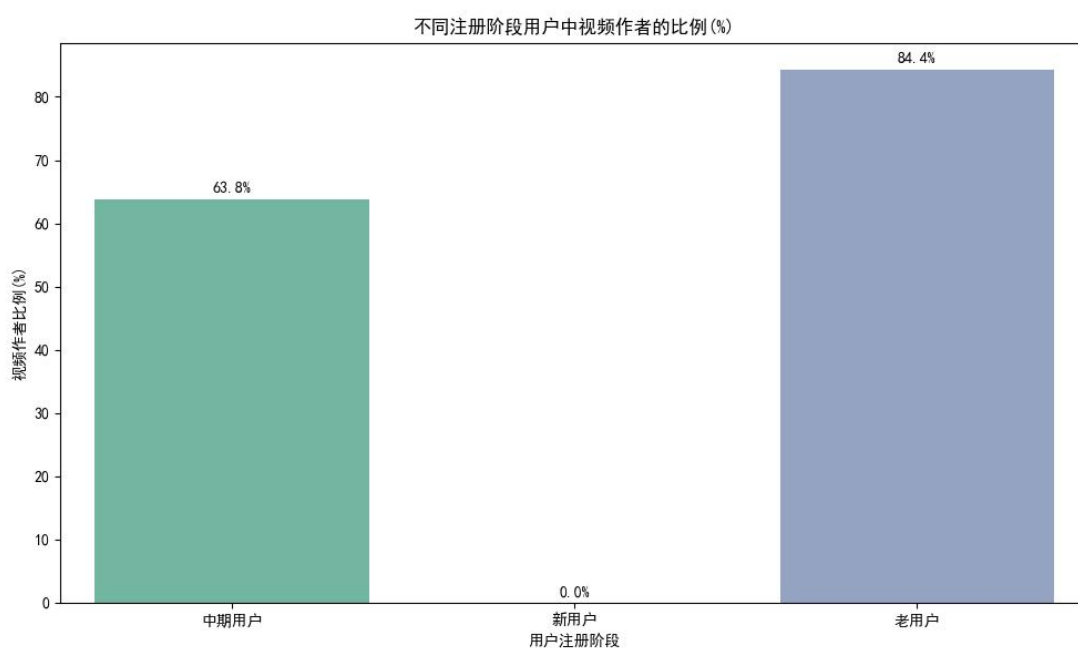


不同用户数数量分布中，能看到仅视频创作者数量（16165）远高于其他类型，说明在该平台里，发过视频的用户群体占比大，普通用户、交叉身份用户数量次之，仅直播主播数量（47）极少，反映出视频创作行为在用户中较为普遍。



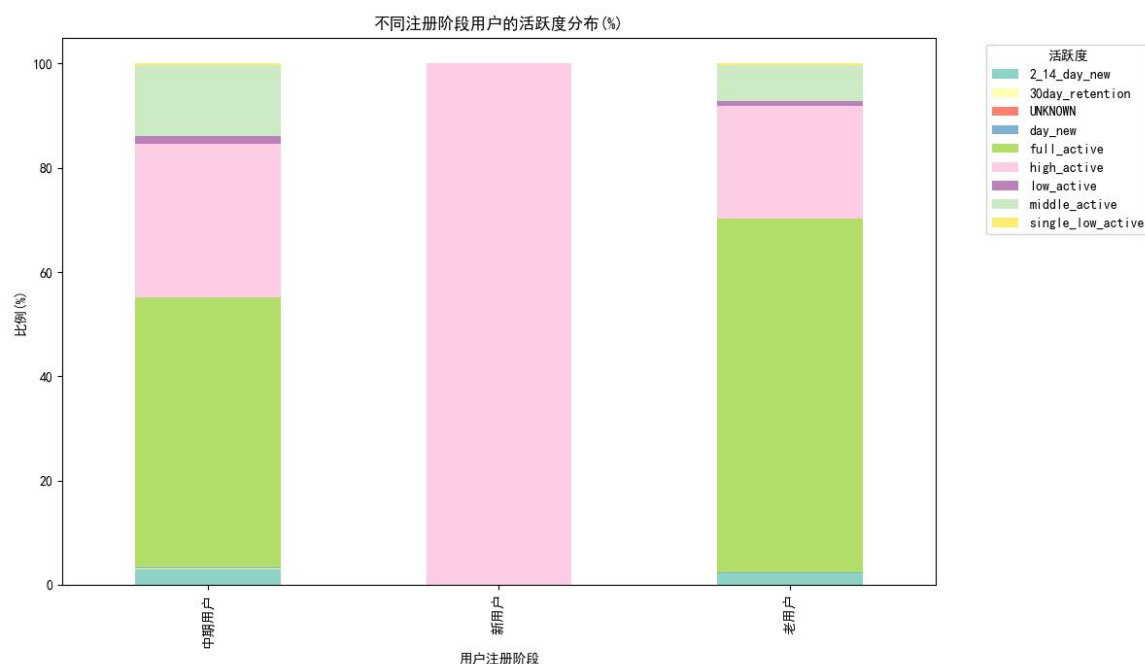
交叉身份用户比值显著高于其他类型，说明其粉丝转化效率（或粉丝增长相对于关注的优势）在四类里最突出，仅视频创作者、仅直播主播依次降低，普通用户比值最低。

### 3. 研究注册时长对用户行为特征的影响



在“不同注册阶段用户中视频作者的比例 (%)”柱状图，实体信息为新用户、中期

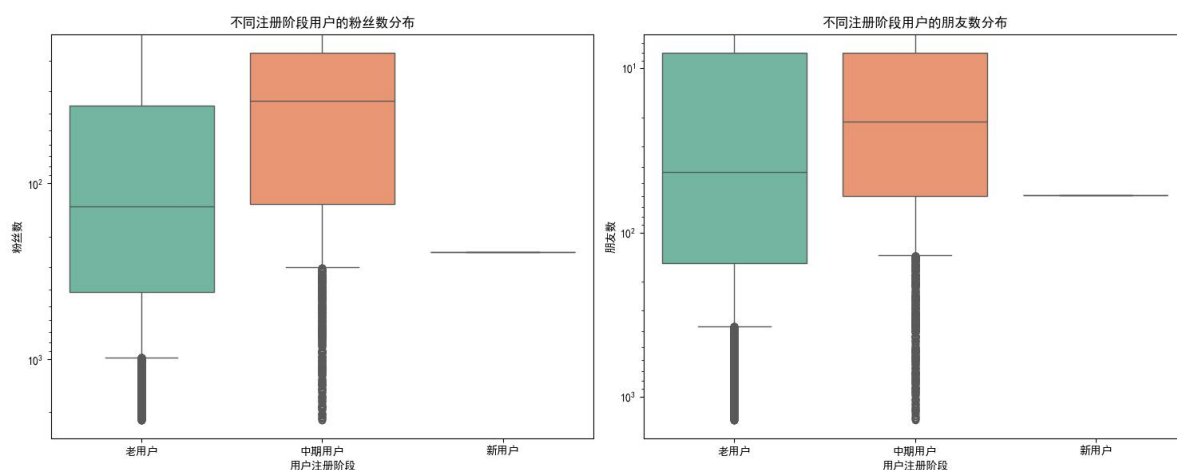
用户、老用户三个注册阶段，及对应视频作者比例：新用户 0.0%、中期用户 63.8%、老用户 84.4%，反映出用户注册阶段越靠后，成为视频作者的比例越高。



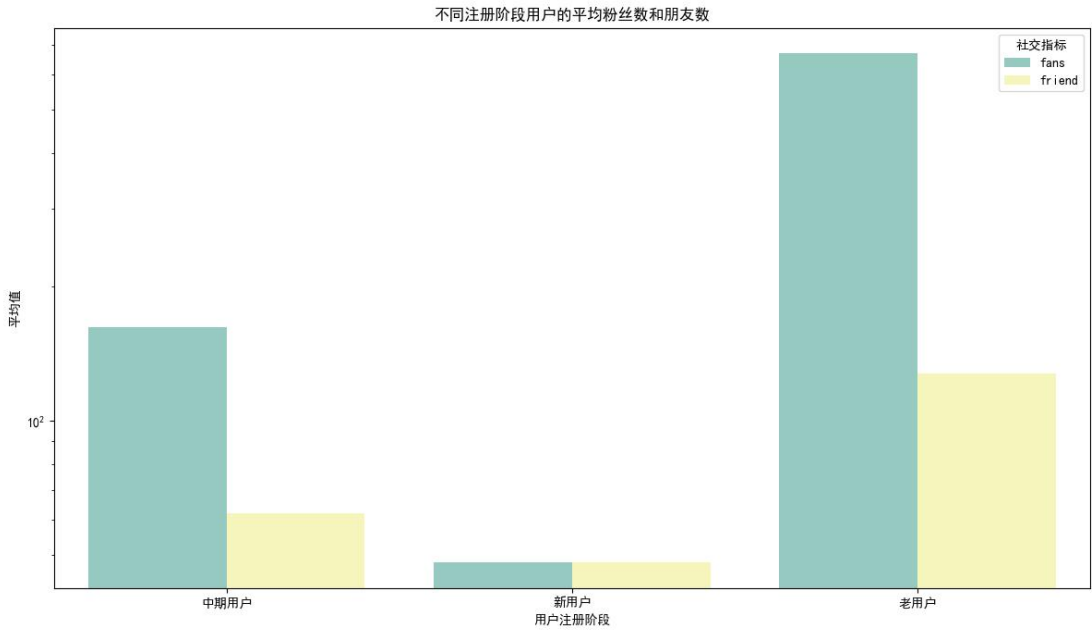
新用户：活跃度相对集中在某些指标上，可能注册后短期内有一定活跃度，但长期来看活跃度分布可能与中期、老用户有差异，反映出其对平台的适应和参与过程刚开始。

中期用户：在多种活跃度指标上有一定占比，说明其活跃度表现较为多样，处于对平台功能和内容深入了解、使用频率逐渐稳定的阶段。

老用户：部分活跃度指标占比较高，显示出其对平台有较高的忠诚度和使用频率，可能是平台的核心用户群体，对平台的依赖度和参与度都较高。整体上，该图有助于分析不同阶段用户的活跃特征，为平台针对不同用户群体制定运营策略提供依据。

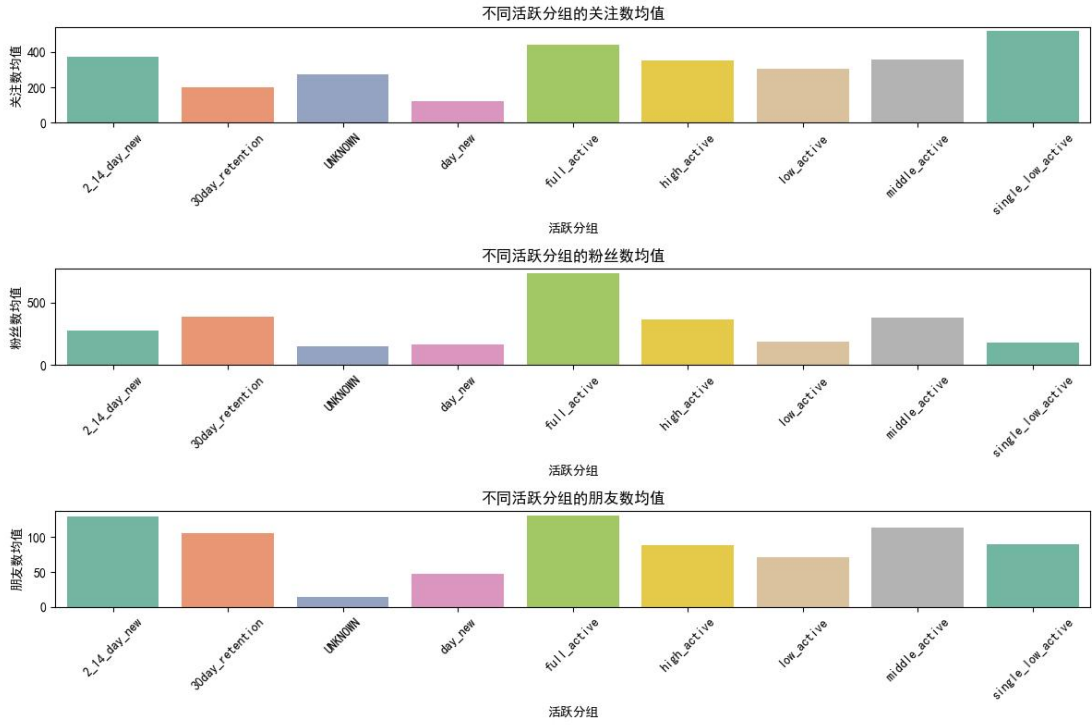


箱线图中，粉丝数分布：新用户粉丝数整体低，中位数、极值均小；中期用户粉丝数中位数、箱体更高，离散大；老用户粉丝数居中，离散度有差异。朋友数分布：新用户朋友数少；中期用户朋友数中位数、箱体高于老用户，离散明显；老用户朋友数有一定基数，离散度不同，反映各阶段社交数据分布特征。



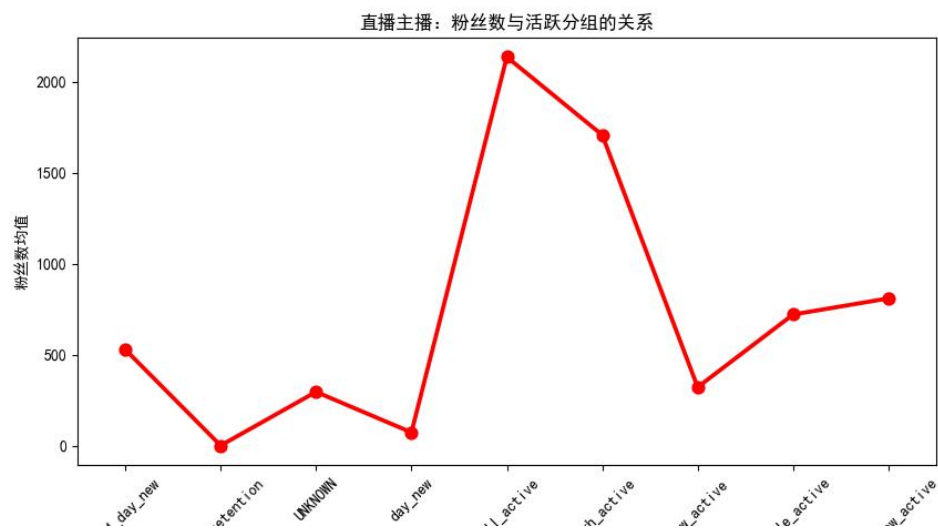
新用户的粉丝数、朋友数均值都很低；中期用户粉丝数均值远高于朋友数；老用户粉丝数均值大幅领先，朋友数均值也较高，整体呈现注册阶段越久，平均粉丝数、朋友数越高的趋势，反映出用户在平台使用时间越长，社交数据积累越多。

#### 4. 用户社交行为的驱动因素分析

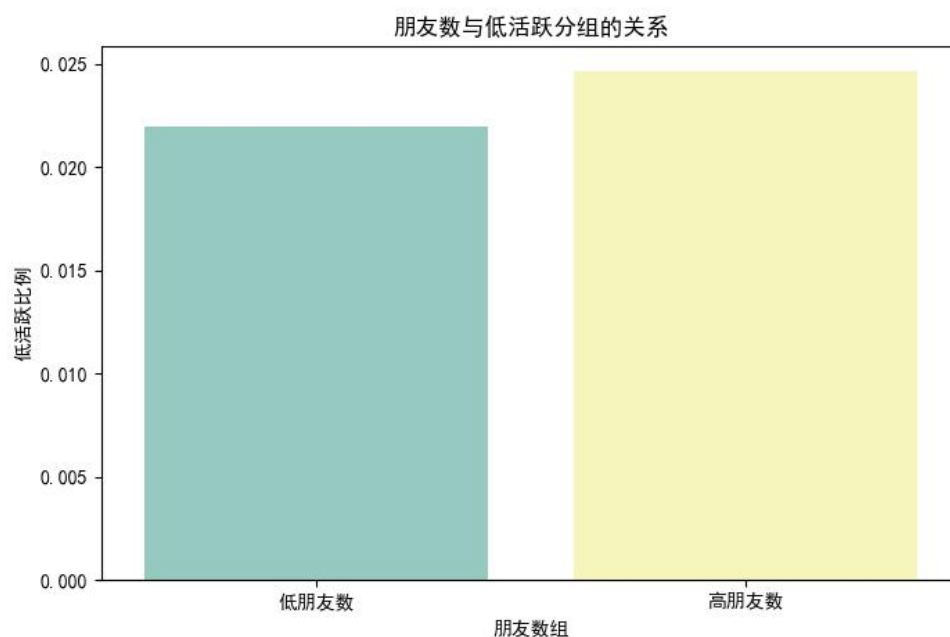


“full\_active” 分组在三项数据均值上普遍突出，“UNKNOWN” 分组数值多较低；不同分组社交数据表现差异大，反映出活跃程度与社交指标间的关联，如高活跃分组（full\_active 等）社交数据均值更高，部分分组（如 single\_low\_active）关注数突出

但粉丝数相对弱。



“full\_active” 分组粉丝数均值峰值显著，远高于其他分组；“30day\_retention”“day\_new” 分组均值极低，不同分组粉丝数均值波动极大，体现活跃分组对直播主播粉丝数影响差异悬殊，高活跃分组（full\_active）助力主播积累更多粉丝。



高朋友数分组的低活跃比例（约 0.025）高于低朋友数分组（约 0.022），说明朋友数多的群体，处于低活跃状态的比例相对更高。

## 四、基于 SkLearn 的短视频平台机器学习推荐模型

### （一）使用模型

本推荐模型主要基于两个小机器学习模型，分别是分类模型 KNN 和聚类模型 k-mean。分类模型主要对视频的质量进行打分，将视频区分成为 1 级、2 级、3 级等不同等级，评估视频质量。KNN 是一种监督学习算法，适用于分类和回归任务，其核心思想是“物以类聚”，即通过计算待预测样本与训练集中所有样本的距离，选取最近的 k 个样本，再通过投票（分类）或取平均值（回归）确定结果。它的优势在于简单易懂、无需训练过程且对异常值不敏感，常见于手写数字识别、推荐系统等场景。

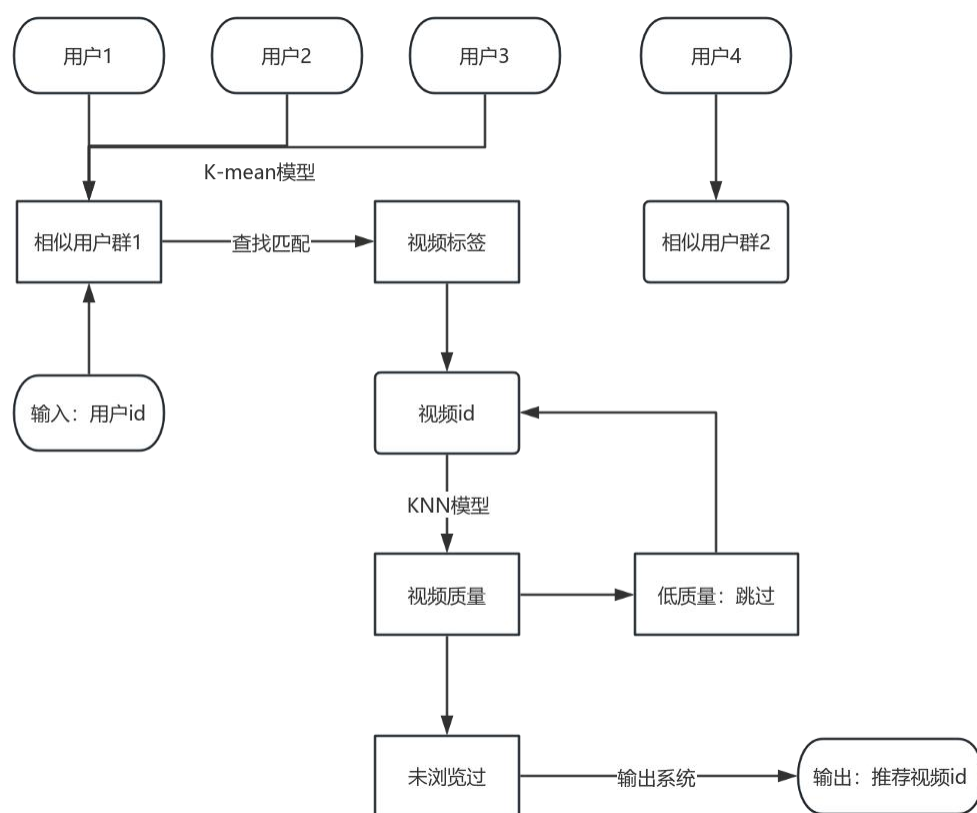


图 1 推荐模型流程图

而聚类模型主要针对用户，将具有相似特征的用户聚为一类，统计这类人群的视频倾向（标签），从而在他们所属的群中推荐共同的视频标签。K-Means 核心是将数据集划分为 k 个簇，使簇内样本相似度高、簇间相似度低。其过程为随机选择 k 个初始聚类中心，计算样本与中心的距离并分配到对应簇，再重新计算簇中心，重复此过程直至中心稳定。该算法原理简单、收敛快，适用于大规模数据集，可用于客户分群、图像分割等。

在将用户进行不同分群，并获得视频标签后，再通过标签（tag）查询对应视频，从而获得该标签下的所有视频 id。随后利用打分系统对视频进行评价，确保推荐的视频

是高质量视频。再获得一定程度的高质量视频后，查询用户的访问记录，将未观看过的视频推送给用户，完成一套推荐流程。

## （二）模型搭建

### 1. K-mean 模型

#### 1) 输入数据模块：

输入主要包括从 SQL 库调取文件、数据预处理、创建聚类特征三个部分。

本数据集主要包括一下数据维度，由 SQL 数据库中关联查询得出。

| #                             | Column             | Non-Null Count | Dtype   |
|-------------------------------|--------------------|----------------|---------|
| ---                           | -----              | -----          | -----   |
| 0                             | user_id            | 5000 non-null  | int64   |
| 1                             | video_id           | 5000 non-null  | int64   |
| 2                             | play_time_ms       | 5000 non-null  | int64   |
| 3                             | video_type         | 4994 non-null  | float64 |
| 4                             | music_type         | 5000 non-null  | float64 |
| 5                             | user_active_degree | 0 non-null     | float64 |
| 6                             | is_live_streamer   | 5000 non-null  | int64   |
| 7                             | is_video_author    | 5000 non-null  | int64   |
| 8                             | follow_user_num    | 5000 non-null  | int64   |
| 9                             | fans_user_num      | 5000 non-null  | int64   |
| 10                            | friends_user_num   | 5000 non-null  | int64   |
| 11                            | tag1               | 5000 non-null  | int64   |
| 12                            | tag2               | 5000 non-null  | int64   |
| 13                            | tag3               | 5000 non-null  | int64   |
| 14                            | user_active_code   | 5000 non-null  | int64   |
| dtypes: float64(3), int64(12) |                    |                |         |
| memory usage: 586.1 KB        |                    |                |         |

表 1 数据类型和数据形式

数据预处理部分将视频标签进行分离，将字符型数据进行编码，编码对应如下：

```
Video_type:          Normal == 1
Video_type:          AD == 0
User_active_degree:  'full_active' == 0
'2_14_day_new' == 1
'high_active' == 2
'middle_active' == 3
'low_active' == 4
```

本研究构建了涵盖四个维度的特征体系：播放行为体系、用户属性特征、内容偏好特征、标签偏好特征。

播放行为特征包括基于 `play_time_ms` 的平均播放时长、总播放时长及基于 `video_id` 去重计数的观看视频数量；用户属性特征选取是否为直播主播、是否为视频创作者、关注用户数、粉丝数、好友数及用户活跃标识，均取分组内首个观测值；内容偏好特征包含基于 `video_type` 均值的视频类型偏好及基于 `music_type` 众数（无众数时取 0）的主导音乐类型；标签偏好特征则通过非零值占比计算三个核心标签的出现频率。所有特征均基于分组数据计算并重置索引，为后续建模提供标准化输入。

## 2) 模型构建

首先从用户特征数据中提取排除 `user_id` 后的特征列，将其转换为数值矩阵；随后使用 `StandardScaler` 进行特征标准化（因聚类算法对特征尺度敏感）；接着应用 `K-Means` 算法（设置聚类数为 `n_clusters`，随机种子 42）执行聚类，为每个用户分配聚类标签；最终返回含聚类结果的用户特征数据及所用的标准化器和聚类模型。

```
def cluster_users(user_features, n_clusters=5):
    """对用户进行聚类"""
    # 提取特征列（排除user_id）
    feature_cols = user_features.columns.drop('user_id')
    X = user_features[feature_cols].values

    # 特征标准化（聚类算法对尺度敏感）
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # 应用K-Means聚类
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    user_features['cluster'] = kmeans.fit_predict(X_scaled)

    return user_features, scaler, kmeans
```

图 2 模型构建代码

## 3) 推荐程序

标签推荐函数通过以下步骤为目标用户生成推荐标签：首先确定目标用户所属聚类；其次筛选出该聚类中的所有用户；然后收集这些用户观看内容的所有非零标签（涵盖 `tag1`、`tag2`、`tag3`）；最后统计标签出现频率并按降序排序，返回前 `N` 个高频标签作为推荐结果。

#### 4) 推荐结果展示

请输入用户 id: 10030

=====

【步骤 1】加载数据中...

=====

【步骤 2】创建用户特征...

=====

【步骤 3】聚类用户中...

=====

【步骤 4】推荐结果

为用户 10030 推荐的标签:

1. 39.0

2. 12.0

3. 6.0

4. 9.0

5. 68.0

## 2. KNN 模型

### 1)输入数据模块

基本同上。

### 2)数据统计描述与热力图

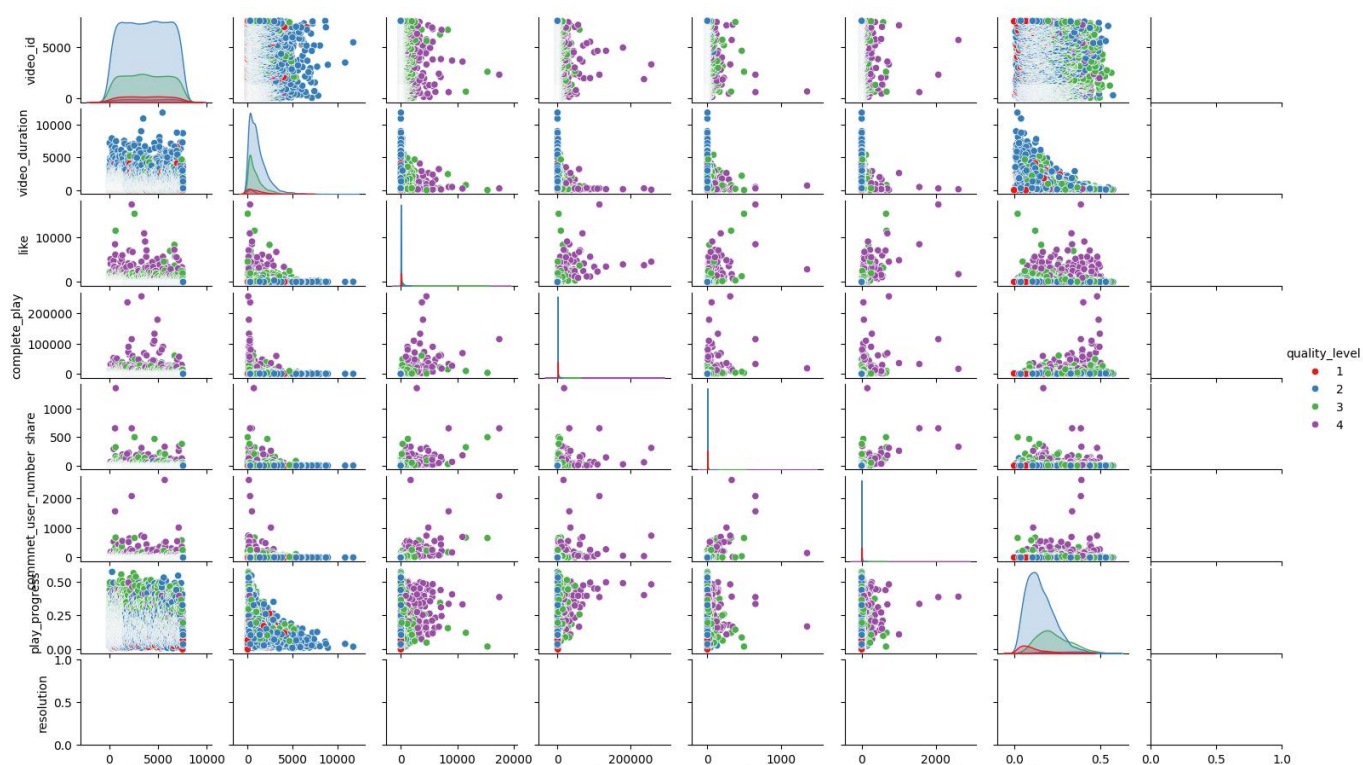


图 3 数据统计描述

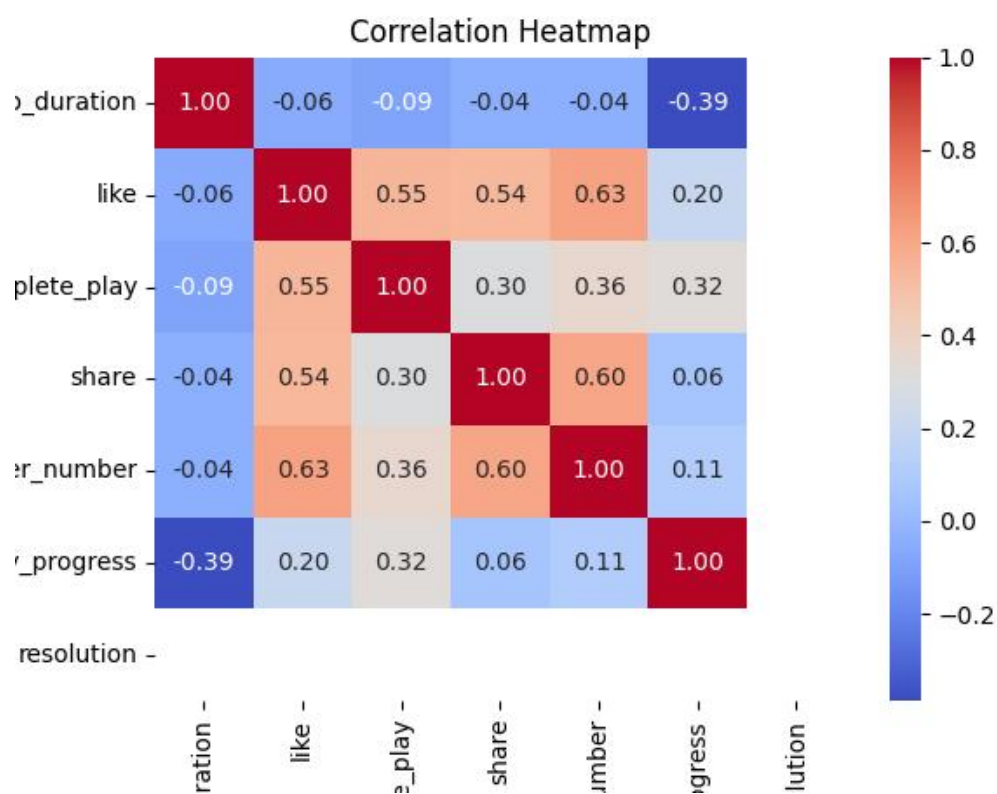


图 4 数据维度热力图

### 3)模型构建

首先加载数据处理模块并获取预处理数据集；通过特征工程阶段，将数据集划分为特征空间  $X$ （剔除 `video_id`、`quality_level` 及 `play_progress` 字段）与目标变量  $y$ （`quality_level`）；采用分层抽样策略按 8:2 比例拆分训练集与测试集，设置 `random_state=42` 以保证实验可复现性。考虑到 KNN 算法对特征尺度的敏感性，使用 `StandardScaler` 进行特征标准化，严格遵循“训练集拟合 - 双集转换”的规范流程；构建  $k=3$  的 K 近邻分类器并在标准化训练集上完成参数学习。

最后使用 `joblib` 包，序列化保存模型参数、标准化器及特征列名至指定路径，其中特征列名的持久化存储为后续预测阶段的特征对齐提供了保障。

```
def load_model_and_scaler():
    """加载保存的模型、标准化器和特征列名"""
    # 加载模型和标准化器
    knn = joblib.load(r'src\sklearn_study\KNN\models\knn_model.pkl')
    scaler = joblib.load(r'src\sklearn_study\KNN\models\scaler.pkl')
    # 加载特征列名
    with open(r'src\sklearn_study\KNN\models\feature_columns.txt', 'r') as f:
        feature_columns = f.read().split(',')
    return knn, scaler, feature_columns

def predict_new_video(new_video_data):
    """预测新视频的质量等级"""
    # 加载模型组件
    knn, scaler, feature_columns = load_model_and_scaler()

    # 确保新视频数据的特征列与训练数据一致
    new_video = pd.DataFrame(new_video_data).reindex(columns=feature_columns, fill_value=0)

    # 标准化新数据
    new_video_scaled = scaler.transform(new_video)

    # 预测
    predicted_level = knn.predict(new_video_scaled)
    return predicted_level[0]
```

图 5 模型加载和预测部分

### 5) 模型测评结果与评估

模型测评结果和评估如下：

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 1            | 0.78      | 0.78   | 0.78     | 80      |
| 2            | 0.93      | 0.95   | 0.94     | 1049    |
| 3            | 0.87      | 0.82   | 0.85     | 352     |
| 4            | 0.81      | 0.72   | 0.76     | 36      |
| accuracy     |           |        | 0.91     | 1517    |
| Macro avg    | 0.85      | 0.82   | 0.83     | 1517    |
| Weighted avg | 0.91      | 0.91   | 0.91     | 1517    |

精确率指预测为某类的样本中实际属该类的比例，召回率指实际为某类的样本中被正确预测的比例，F1-score 为两者的调和平均，support 为该类实际样本数。模型整体准确率达 91%，表现较好，加权平均指标亦为 91%，说明在占比高的类别上表现稳定；各等级中，2 级视频（样本量最大，1049 个）表现最佳（F1-score 94%），3 级视频（352 个样本）表现良好（F1-score 85%），而 1 级（80 个样本）和 4 级（36 个样本）表现较弱，F1-score 分别为 78%、76%，尤其 4 级因样本最少，召回率仅 72%。改进方向包括解决数据不平衡问题（如增加 4 级样本或过采样）、补充区分度特征、优化低质量相关特征维度、调整 KNN 参数或尝试其他模型（如随机森林），以提升小众类别的识别效果。

### 3. KNN 和 K-mean 结合使用的预测模型

#### 1)模型主要流程

模型第一步初始化与用户兴趣建模，以用户 ID 为输入，通过 K-mean 模型的 lunch\_recommendation 模块生成用户兴趣标签列表（tag\_list），构建用户兴趣画像，作为后续内容匹配的核心依据。然后基于兴趣标签，视频候选集生成与数据预处理，通过 find\_videos 组件从数据库提取关联视频的多维度特征（基础属性：分辨率、时长；交互特征：点赞、完播、分享、评论数；标签特征），并执行标准化处理（分辨率格式统一、时长单位转换、标签列拆分与格式化），形成结构化候选视频池。同时通过 SQL 查询用户历史观看记录，构建去重过滤规则。

随后，调用 load\_model\_and\_scaler 加载预训练的 KNN 质量等级预测模型、特征标准化器及特征列名映射表，确保推理环境一致性。通过 predict\_new\_video 组件对候选视频执行特征对齐、标准化处理及质量等级预测（1-4 级），形成视频质量量化指标。

最后推荐决策与结果输出，采用分层推荐策略：优先筛选质量等级 > 3 且未观看

的视频；若不满足则选取该标签下质量评分最高的未观看视频。最终按兴趣标签维度组织推荐结果，输出每个标签对应的最优视频 ID，实现兼具用户兴趣匹配与质量保障的个性化推荐。

模型输出展示：

请输入用户 id: 10030

=====

【步骤 1】加载数据中...

=====

【步骤 2】创建用户特征...

=====

【步骤 3】聚类用户中...

=====

【步骤 4】推荐结果

为用户 10030 推荐的标签：

1. 39.0

2. 12.0

3. 6.0

4. 9.0

5. 68.0

=====

加载视频数据库中...

正在根据视频数据库和视频质量为用户推荐优质视频...

推荐用户 10030 的视频为：

类别 39 推荐视频 ID: 453

类别 12 推荐视频 ID: 38

类别 6 推荐视频 ID: 6241

类别 9 推荐视频 ID: 140

类别 68 推荐视频 ID: 1595

## 2)具象化展示模块

一个简易的输入和展示模块，包括展示框、思考进度条、思考状态等。



图 6 具象化模块展示

## （三）结论

本研究构建了基于 K-Means 聚类模型与 KNN 分类模型的视频个性化推荐系统，整体流程完整且具有实际应用价值。

系统融合了两种经典机器学习模型：K-Means 聚类模型用于用户分群，通过提取用户播放行为、属性、内容偏好及标签偏好四大维度特征，将具有相似特征的用户聚为一类，进而生成反映群体兴趣的标签推荐；KNN 分类模型用于视频质量评估，通过学习视频基础属性（分辨率、时长）与交互特征（点赞、完播等），将视频质量划分为 1-4 级，实现对视频质量的量化分级。

在模型精度方面，KNN 分类模型表现优异，整体准确率达 91%，对样本量较大的 2 级（F1-score 94%）和 3 级视频（F1-score 85%）识别精准，虽对样本量较少的 1 级（F1-score 78%）和 4 级视频（F1-score 76%）存在一定误差，但整体满足质量评估需求；K-Means 聚类模型通过群体兴趣标签的高频统计，能有效捕捉用户偏好，从推荐结果来看，生成的标签与用户历史行为匹配度较高，为后续视频匹配提供了可靠依据。

## （四）讨论与总结

该推荐系统融合 K-Means 与 KNN 模型的优势，精度达标、易用性强且适配性良好，具备较高的实际应用价值。

模型的实用性与易用性较高，从用户 ID 输入到兴趣标签生成，再到视频质量筛选

与去重推荐，步骤清晰且自动化程度高；同时，K-Means 与 KNN 均为原理简单、部署成本低的算法，无需复杂的算力支持，便于实际场景中的快速应用与维护。模型能较好适应多样化需求：通过用户聚类可覆盖不同兴趣特征的用户群体，实现个性化标签推荐；视频质量分级可适配从低到高的多等级视频评估，兼顾主流质量视频（2 级、3 级）与小众质量视频（1 级、4 级）；此外，系统通过过滤用户已观看视频，避免重复推荐，进一步提升了对实际场景的适配性。

在以后的研究中，可以在算法层面引入 DBSCAN 或谱聚类优化用户分群，采用随机森林、XGBoost 等模型提升视频质量分级精度，尤其改善 1 级、4 级等小众类别的识别效果，同时结合用户 - 视频嵌入技术捕捉深层兴趣关联；特征层面，增加用户行为时序特征与视频内容语义特征，并建立动态更新机制以响应兴趣变化；推荐策略上，平衡精准性与多样性，引入场景感知与冷启动优化；工程上，搭建实时推荐引擎并完善多维度评估体系。此外，需重点解决当前数据量不足的问题（如 4 级视频样本仅 36 个），通过扩充高质量样本或采用过采样技术，为模型提供更充分的学习基础，进一步提升系统的稳定性与泛化能力。

## 五、视频每小时点赞量混合预测模型

### （一）模型核心目标

该模型核心目标为对视频每小时互动峰值和总量开展时间序列预测，一方面，依托历史 2 周小时级点赞数据，完成未来 24 小时每小时点赞量的短期预测，并与真实数据对比验证模型效果，同时在此基础上，额外延伸预测后续 24 小时数据，实现“24 小时验证 + 24 小时延伸”的 48 小时连续预测；另一方面，同步识别预测周期内的点赞量峰值，涵盖 24 小时验证期的预测峰值、真实峰值，以及 24 小时延伸期的预测峰值，为流量调度、内容推荐等业务决策提供参考。

### （二）使用模型和模型介绍

模型主要遵循“数据获取→数据预处理→时间序列分解→多模型训练→混合预测→结果可视化”的流程，具体步骤如下

#### 1. 1.时间序列分解

采用季节性分解（seasonal\_decompose）将每小时点赞量数据拆分为趋势项（长期变化规律）、周期项（24 小时日内波动）和残差项（随机波动）。通过对比残差方差，自动选择加法或乘法分解模型，适配不同数据波动特性。

#### 2. 2.SARIMA 模型

针对分解后的“趋势 + 周期”组合项建模，捕捉时间序列的长期趋势和日内季节性。通过自回归（AR）、差分（I）、移动平均（MA）及季节性组件，预测未来 48 小时的趋势与周期叠加结果。

### 3. 3.随机森林回归

对分解后的残差项建模，利用小时、是否周末、星期几等时间特征，学习随机波动规律，修正 SARIMA 的预测误差。通过 100 棵决策树（最大深度 7）平衡拟合精度与泛化能力。

### 4. 4.混合预测策略

结合 SARIMA 的趋势周期预测与随机森林的残差预测，根据分解类型（加法 / 乘法）进行结果融合，确保预测值非负（点赞量不可能为负），最终输出 48 小时连续预测结果。

## （三）模型构建主要流程

### 步骤 1：数据获取与分割（get\_data 函数）

#### 1. SQL 查询提取原始数据

关联两张业务表：log\_standard（用户行为日志表）：获取 user\_id（用户 ID）、video\_id（视频 ID）、date（日期）、hourmin（时分）、time\_ms（时间戳，毫秒级）；以及 video\_features\_statistic\_pure（视频特征表）：获取 video\_id 对应的 like\_cnt（点赞量）；

通过 video\_id 内连接两表，确保每个用户行为日志匹配对应的视频点赞量。

#### 2. 数据预处理

- o 类型转换：time\_ms 转为数值型，再通过 pd.to\_datetime(..., unit='ms')转为可读时间（readable\_time），并按小时取整得到 hour（小时级时间索引）；

- o 异常值处理：like\_cnt 转为数值型，缺失值填充为 0（无点赞记录视为 0 点赞）；

- o 小时级聚合：按 hour 分组求和 like\_cnt，得到“每小时总点赞量”时间序列，并通过 asfreq('H', fill\_value=0)补全缺失的小时时间点（确保时间序列连续）。

该模型的训练集与测试集分割以输入的 train\_cutoff（训练截止时间，默认值为 2022-05-07 00:00:00）为核心基准：其中训练集涵盖 train\_cutoff 前 2 周至 train\_cutoff 的所有小时级数据，共  $2 \times 7 \times 24 = 336$  小时，确保能覆盖 2 个完整的日内周期以支撑模型学习周期性规律；测试集则选取 train\_cutoff 后 1 小时至 24 小时的小时级数据，共 24 小时，专门用于模型预测效果的验证；同时，流程中设置了数据完整性检查机制，若训练

集为空或测试集实际时长不足 24 小时，将直接抛出异常提示，保障后续建模数据的有效性。

## 步骤 2：混合模型训练与预测（train\_hybrid\_model 函数）

该步骤是模型核心，分为“时间序列分解→SARIMA 训练→随机森林训练→混合预测”4 个子环节：

### 时间序列分解（加法 / 乘法自动选择）

1. **数据适配**：为避免乘法分解中 0 值导致的计算错误，对训练集数据加 1（ $\text{train\_data\_adj} = \text{train\_data} + 1$ ）；
2. **双模型分解**：分别用加法（additive）和乘法（multiplicative）模型分解数据，得到两组“趋势 + 周期 + 残差”；
3. **最优分解选择**：计算两组分解结果的残差方差，选择残差方差更大的模型（更能反映数据波动，便于后续残差修正）；
4. **结果还原**：将分解后的趋势、周期、残差与原始训练集时间索引对齐，若为乘法模型，需将残差减 1 以还原原始数据尺度。

### 模型一：SARIMA 模型训练（趋势 + 周期预测）

#### 1. 模型参数设置：

o 基础参数（ $\text{order}=(1,1,1)$ ）：AR (1) 自回归项、I (1) 差分（确保序列平稳）、MA (1) 移动平均项；

o 季节性参数（ $\text{seasonal\_order}=(2,1,1,24)$ ）：季节性 AR (2)、季节性差分 I (1)、季节性 MA (1)、周期长度 24（日内季节性）；

3. **模型训练**：基于分解后的“趋势 + 周期”组合项训练 SARIMA 模型，禁用平稳性 / 可逆性强制约束（适配实际数据特性）；

4. **趋势 + 周期预测**：用训练好的 SARIMA 模型预测未来 48 小时（24 小时测试 + 24 小时延伸）的“趋势 + 周期”值。

### 模型二：随机森林训练（残差预测）

#### 1. 特征工程：构建残差预测的特征集，包含 3 类时间特征：

ohour：小时（0-23，捕捉日内波动）；

ois\_weekend: 是否周末 (0 = 工作日, 1 = 周末, 捕捉周度波动);

oday\_of\_week: 星期几 (0-6, 捕捉周内差异);

2. **模型训练**: 用残差作为目标变量, 时间特征作为输入, 训练随机森林回归模型 (100 棵树, 最大深度 7, 确保模型拟合且不过拟合);

3. **残差预测**: 基于未来 48 小时的时间特征, 用训练好的随机森林预测对应残差。

## 混合预测 (结果组合)

根据之前选择的分解模型 (加法 / 乘法), 组合 SARIMA 的 “趋势 + 周期” 预测值与随机森林的残差预测值:

加法模型: 混合预测值 = SARIMA 预测值 + 残差预测值;

乘法模型: 混合预测值 = SARIMA 预测值  $\times$  (1 + 残差预测值);

边界处理: 用 clip(min=0) 确保预测值非负 (点赞量不可能为负)。

最终将预测结果拆分为 “24 小时测试期预测” 和 “24 小时延伸期预测”, 并识别两个周期内的点赞峰值 (时间 + 数值)。

效果展示:

选择加法模型进行时间序列分解

trend 非零样本数: 337

seasonal 非零样本数: 337

residual 非零样本数: 337

residual 样本预览: [12297.438645514834, -52085.34075057176, 117245.51252644986, 279504.83115407574, -64938.41111715231, 259098.28554015368, 273298.46117136965, 132073.995833086, 359028.9938574579, 155858.57884031715]

=== 对比部分 ===

预测峰值: 2022-05-07 13:00, 点赞量: 1333030

真实峰值: 2022-05-07 11:00, 点赞量: 1076635

=== 延伸预测部分 ===

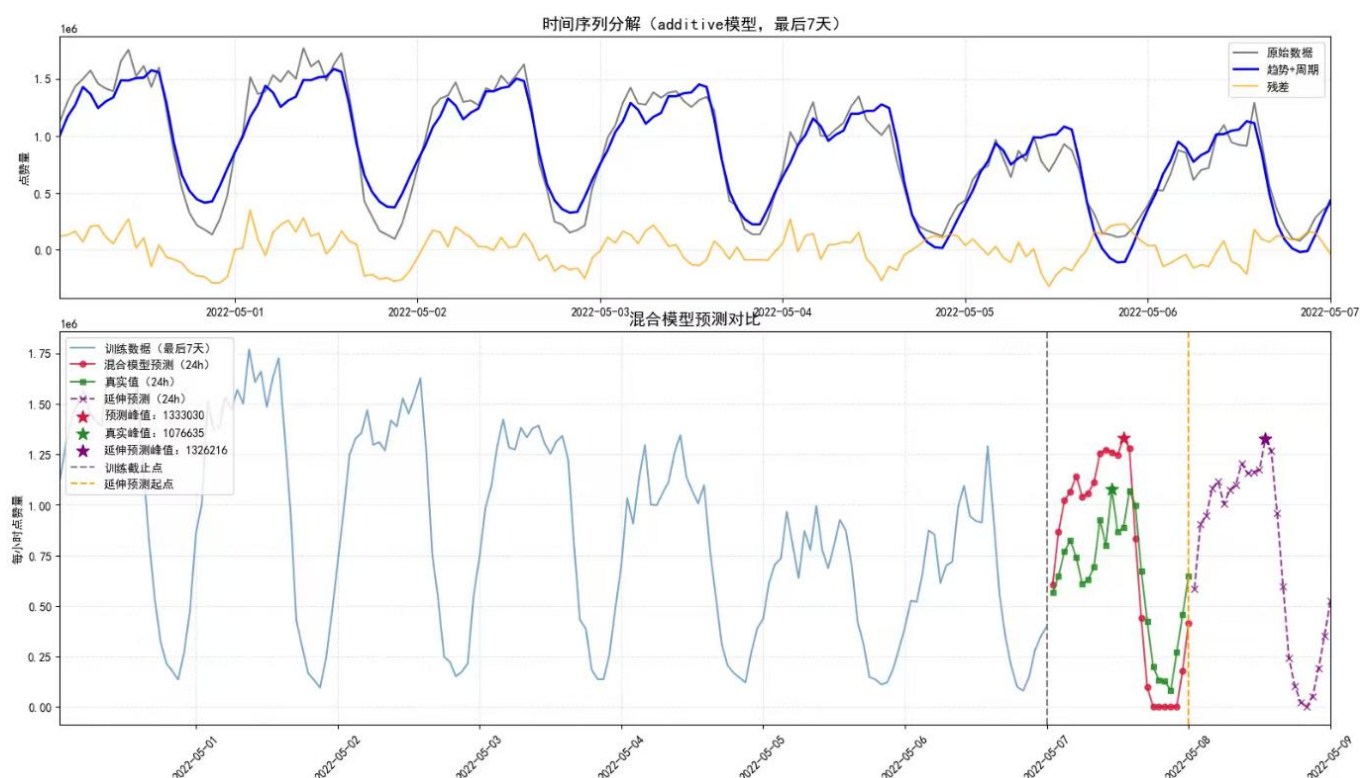
延伸预测峰值：2022-05-08 13:00，点赞量：1326216

### 步骤 3：结果可视化与输出（plot\_results 函数 + 主程序）

1.可视化输出：模型生成包含 2 个子图的综合图表。子图 1（时间序列分解）：展示训练集最后 7 天的原始数据、“趋势 + 周期”、残差，验证分解效果；子图 2（预测对比）：展示训练集最后 7 天数据、测试期真实值、测试期预测值、延伸期预测值，并标记各周期峰值和关键时间节点（训练截止点、延伸预测起点）；

2.文本输出：打印关键结果（测试期预测峰值 vs 真实峰值、延伸期预测峰值），便于量化评估模型效果。

效果展示：



该可视化结果包含时间序列分解与混合模型预测对比两部分。上图基于加法模型对最后 7 天数据分解，灰色原始数据呈现每小时点赞量日内波动，蓝色趋势 + 周期线提取常规变化模式，黄色残差体现随机波动；下图展示模型预测表现，浅蓝色为训练最后 7 天数据，24 小时验证期（2022-05-07）内，混合模型预测峰值数值为 1333030，真实峰值数值为 1076635，预测值高于真实值，误差绝对值约 256395，约为真实值的 24%。但红色预测曲线与绿色真实值曲线整体趋势高度贴合，可捕捉日内多峰波动；延

伸预测期（2022-05-08 及之后），紫色延伸预测峰值数值为 1326214，呈现后续 24 小时走势，两个预测值之间有显著的不同，说明第二次的预测有根据之前的修复倾向，并且也更符合长期的数据走向和整体趋势。

#### （四）讨论与总结

该模型的核心优势体现在多维度技术特性的协同优化：其一，采用混合建模架构，有机融合 SARIMA 模型对时间序列“趋势-周期”耦合特征的强刻画能力，与随机森林对残差序列“非线性波动”的高效拟合能力，有效突破单一模型（纯 SARIMA 或纯机器学习模型）在复杂时序规律捕捉上的局限性，显著提升预测精度；其二，具备数据自适应能力，通过残差方差量化对比机制，自动遴选加法或乘法分解模型，无需人工参数干预即可动态适配不同场景下的数据波动特性；其三，在长期预测任务中，模型可规避单一趋势的固化输出问题，依托时序规律的动态学习机制实现自我调节，确保在时间维度上持续维持较高的预测准确度。

## 附录

代码（gitee）：[https://gitee.com/jss01\\_1/sjfx\\_qzc.git](https://gitee.com/jss01_1/sjfx_qzc.git)

作者 gitee：<https://gitee.com/zichenqin>

Python 版本：Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32

Mysql 版本（基于 docker 的 Linux 虚拟系统）：8.0.28

启动 Mysql：`docker exec -it Zichen-server mysql -uroot -p20051108`

Navicat15 版本：15.0.19